
Green Vehicle Routing Problem

**with Fuel Consumption Minimization
Using a Genetic Algorithm**

**Final Report
EMU676 Optimization Models and Algorithms in
Transportation and Distribution**

Muhammed Reha Özdemir – “N25220211”

June 3, 2026

Hacettepe University

Contents

1	Introduction	1
2	Problem Definition	2
2.1	General Description	2
2.2	Operational Constraints and Objectives	2
3	Mathematical Model	3
3.1	Sets and Parameters	3
3.2	Decision Variables	3
3.3	Objective Function	3
3.4	Constraints	4
4	Methodology: Genetic Algorithm	5
4.1	Why Genetic Algorithm?	5
4.2	General Framework and Algorithmic Steps	5
5	Solution Method	7
5.1	Chromosome Encoding	7
5.2	Decoding: From Permutation to Vehicle Routes	7
5.3	Fitness Evaluation	8
5.4	Initial Population	9
5.5	Selection: Tournament	10
5.6	Crossover: Order Crossover (OX)	10
5.7	Mutation: Swap and Inversion	11
5.8	Elitism and Replacement	12
5.9	Stopping Criterion	12
5.10	Parameter Choices	12
5.11	A Toy Example	13
6	Computational Study	15
6.1	Test Instances	15
6.2	Experimental Setup	16
6.3	Computational Results	17
6.4	Convergence Behaviour	18
6.5	Discussion	19
7	Conclusion	20
	Bibliography	21

Abstract

This report presents the complete work on a project about the Green Vehicle Routing Problem (GVRP), a variant of the classical Capacitated Vehicle Routing Problem (CVRP) where the objective is to minimize fuel consumption instead of total distance. The key idea, taken from Xiao et al. [1], is that the fuel consumption rate of a vehicle depends linearly on its current load, so the sequence in which customers are visited has a direct effect on total fuel used. A Mixed Integer Linear Programming (MILP) formulation is given based on the model of Xiao et al. [1], covering sets, parameters, decision variables, objective function and constraints. A Genetic Algorithm (GA) with permutation-based encoding, Order Crossover, swap and inversion mutation, tournament selection and elitism is implemented in Python and tested on ten GVRP instances of varying sizes (from 10 to 150 customers). Each instance is solved five times with different random seeds, and the GA is also compared against a deterministic nearest-neighbor greedy baseline. The GA finds the exact optimum on the smallest instance (verified by full enumeration of $10!$ permutations) and beats the greedy baseline on the small and medium instances, but underperforms it on the largest instances. This is consistent with the well-known observation that a pure GA without local search tends to lose against simple route-construction heuristics on large VRPs.

1 Introduction

Fuel is one of the biggest costs in road transportation. According to Xiao et al. [1], fuel cost can make up more than 60% of total transportation costs for some companies, which means that even a small improvement in fuel efficiency can save a lot of money over time. Besides the cost side, reducing fuel consumption also helps with lowering CO₂ emissions, and this has become more and more relevant because of environmental regulations. Demir et al. [6] note in their review that road freight transportation is one of the main contributors to greenhouse gas emissions, which is why there has been a clear shift in operations research toward what is now usually called green logistics.

The Green Vehicle Routing Problem (GVRP) tries to deal with this by changing the objective of the classical VRP from minimizing distance to minimizing fuel consumption. At first this might seem like a small change, but it is actually quite significant. Fuel consumption is not just a function of distance — it also depends on how heavy the vehicle is at different points along the route. So two routes with the same total distance can have different fuel costs depending on the visiting order. Vidal et al. [3] survey many VRP variants including the GVRP and show that problems like this have received a lot of attention recently, mostly because they reflect real operational concerns better than simpler distance-based models.

The term GVRP itself covers more than one type of problem. Lin et al. [7] group the literature into three main categories: the green-VRP in a narrow sense, which deals with alternative-fuel vehicles and limited driving range (as in Erdoğan and Miller-Hooks [8]); the pollution-routing problem (PRP) introduced by Bektaş and Laporte [5], which considers both load and speed effects on emissions; and VRP variants in reverse logistics. The model used in this project follows the Fuel Consumption VRP (FCVRP) of Xiao et al. [1], which sits closer to the PRP family: it captures the load-dependent part of fuel consumption (like the PRP) but leaves out the speed and time-window aspects to keep the formulation cleaner. A more recent review by Asghari and Mirzapour Al-e-hashem [2] reports that metaheuristics, and in particular genetic algorithms, have become the standard for solving medium and large GVRP instances, since exact methods generally cannot scale well beyond a few dozen customers.

In this project, the GVRP is studied in its single-depot form: one depot, a set of customers, and a fleet of identical vehicles. The contribution of the project is to implement a Genetic Algorithm tailored to the load-dependent fuel objective of Xiao et al. [1], evaluate it on a set of test instances of varying sizes, and study its convergence behaviour, the variability of its solutions across runs, and its performance against a simple greedy baseline. The rest of the report is organised as follows. Section 2 describes the problem. Section 3 gives the mathematical formulation. Section 4 explains the Genetic

Algorithm at a high level. Section 5 describes the solution method in detail, component by component. Section 6 presents the test instances, the experimental setup and the computational results. Section 7 concludes the report.

2 Problem Definition

2.1 General Description

The problem setting is as follows. There is one depot and n customers. A fleet of m homogeneous vehicles with capacity Q starts and ends at the depot. Every customer has a known demand and must be visited exactly once by one vehicle. The goal is to find vehicle routes that minimize total fuel consumption.

What makes GVRP different from the classical CVRP is the objective. In CVRP, the aim is to minimize total distance traveled. But in GVRP, we want to minimize fuel consumption, and fuel depends not only on distance but also on how much the vehicle is carrying at each point in the route. As shown by Xiao et al. [1], a vehicle carrying heavier loads consumes more fuel per kilometer, so the order in which customers are visited matters even within a single route. This means that two routes with the same total distance can have very different fuel costs depending on the sequence of customer visits. A small visual illustration of this effect on a 5-customer example is given in Section 5.11.

2.2 Operational Constraints and Objectives

The main constraints of the problem are:

- Each customer must be visited by exactly one vehicle.
- Every vehicle starts and ends its route at the depot.
- The total demand served by a single vehicle cannot exceed its capacity Q .
- The number of vehicles used cannot exceed m .

The objective is to minimize total fuel consumption across all routes. Following Xiao et al. [1], the fuel consumption rate on any arc depends linearly on the vehicle's current load, which means that serving customers with higher demands earlier in a route generally leads to lower overall fuel consumption.

3 Mathematical Model

The model presented in this section directly follows the FCVRP formulation of Xiao et al. [1]. We use their notation as closely as possible to keep the connection to the original paper clear.

3.1 Sets and Parameters

Let $V = \{0, 1, \dots, n\}$ be the set of all nodes, where node 0 is the depot and nodes 1 to n are customers. Let $C = \{1, \dots, n\}$ denote the customer set.

The parameters are defined as follows:

- n : number of customers
- m : number of available vehicles
- d_{ij} : distance between nodes i and j
- D_i : demand of customer i ($D_0 = 0$ for the depot)
- Q : vehicle capacity
- F : fixed cost per vehicle used
- c_0 : unit fuel cost
- ρ_0 : fuel consumption rate (FCR) of an empty vehicle per unit distance
- ρ^* : FCR of a fully loaded vehicle per unit distance
- $\alpha = (\rho^* - \rho_0)/Q$: the slope of the linear FCR function, i.e., how much FCR increases per unit of additional load

3.2 Decision Variables

- $x_{ij} \in \{0, 1\}$: equals 1 if the vehicle travels directly from node i to node j , 0 otherwise
- $y_{ij} \geq 0$: the load (weight of cargo) carried by the vehicle when traveling from node i to node j

3.3 Objective Function

The objective function minimizes the sum of fixed vehicle costs and total fuel costs across all routes:

$$\min H = \sum_{j=1}^n F x_{0j} + \sum_{i=0}^n \sum_{j=0}^n c_0 d_{ij} (\rho_0 x_{ij} + \alpha y_{ij}) \quad (1)$$

The first term is the total fixed cost of the vehicles used. The second term represents the total fuel cost, where $\rho_0 x_{ij}$ accounts for the baseline fuel use of an empty vehicle and αy_{ij} adds the load-dependent part. Since d_{ij} , c_0 , ρ_0 , and α are all constants, the objective function is linear in the decision variables.

3.4 Constraints

$$\sum_{j=0}^n x_{ij} = 1, \quad i = 1, \dots, n \quad (2)$$

$$\sum_{j=0}^n x_{ij} - \sum_{j=0}^n x_{ji} = 0, \quad i = 0, \dots, n \quad (3)$$

$$\sum_{\substack{j=0 \\ j \neq i}}^n y_{ji} - \sum_{\substack{j=0 \\ j \neq i}}^n y_{ij} = D_i, \quad i = 1, \dots, n \quad (4)$$

$$y_{ij} \leq Q x_{ij}, \quad i, j = 0, \dots, n \quad (5)$$

$$\sum_{j=1}^n x_{0j} \leq m \quad (6)$$

$$y_{ij} \geq 0, \quad i, j = 0, \dots, n \quad (7)$$

$$x_{ij} \in \{0, 1\}, \quad i, j = 0, \dots, n \quad (8)$$

Constraint (2) ensures that each customer is visited exactly once. Constraint (3) is the flow conservation for routing: any vehicle that arrives at a node must also leave from that node, which guarantees route continuity and forces vehicles to return to the depot. Constraint (4) is the load flow conservation: the load arriving at customer i minus the load leaving after the visit equals the demand D_i , meaning the cargo decreases by D_i after each delivery. This constraint also eliminates subtours because a subtour would not be connected to the depot and could not satisfy the load balance. Constraint (5) links the load variables to the routing variables and enforces the capacity limit: if arc (i, j) is not used, no cargo can be on it. Constraint (6) limits the number of vehicles leaving the depot to at most m , which models the available fleet size. Constraint (7) is the non-negativity of the load variables, which is needed because y_{ij} represents a physical quantity (the weight of cargo on the arc) and cannot be negative. Finally, constraint (8) defines the binary nature of the routing variables.

4 Methodology: Genetic Algorithm

4.1 Why Genetic Algorithm?

Since the GVRP is NP-hard, exact methods can only solve very small instances in reasonable time. For anything larger, a metaheuristic is needed. Among the available options, a Genetic Algorithm (GA), originally introduced by Holland [9], was selected for this project.

GA works well for combinatorial problems like VRP because solutions can be naturally represented as permutations of customers, and the crossover and mutation operators can be applied directly to these permutations without any complicated transformation. According to Asghari and Mirzapour Al-e-hashem [2], GA is one of the most commonly used methods for GVRP variants in the literature, so there is plenty of reference material to guide the implementation.

There is also a practical side to this decision. GA can be built and tested step by step — each component like initialization, crossover, and mutation can be checked independently before putting everything together. This makes it easier to find and fix problems during development, especially compared to something like Ant Colony Optimization where several parts need to work correctly at the same time from the start.

4.2 General Framework and Algorithmic Steps

The algorithm keeps a population of candidate solutions and updates it over many generations. Each solution is stored as a permutation of customer indices — for example $[3, 1, 5, 2, 4]$ for five customers. To get actual vehicle routes from a permutation, we go through it from left to right and keep adding customers to the current vehicle's route until the next one would exceed the capacity Q , then we open a new route. This decoding step is simple and works well with the genetic operators.

The overall structure of the algorithm is shown in Algorithm 1.

Algorithm 1 Genetic Algorithm for GVRP

```
1: Generate initial population  $P$  of size  $N_{pop}$  (random + greedy seeds)
2: Decode each individual and evaluate its fitness using Eq. (1)
3:  $best \leftarrow$  best individual found so far
4: while stopping criterion not met do
5:   Copy the top  $e$  individuals of  $P$  into the new population (elitism)
6:   while new population size  $< N_{pop}$  do
7:     Select two parents using tournament selection of size  $k$ 
8:     With probability  $p_c$ , apply Order Crossover (OX); otherwise, copy
       parent 1
9:     With probability  $p_m$ , apply swap or inversion mutation
10:    Add the offspring to the new population
11:  end while
12:  Replace  $P$  with the new population
13:  Re-evaluate fitness and update  $best$  if a better solution is found
14: end while
15: return  $best$ 
```

The main steps of Algorithm 1 are explained line by line below.

- **Line 1:** An initial population of N_{pop} chromosomes is generated. About 80% of the individuals are random permutations of the customer set C , and the remaining 20% come from a nearest-neighbor greedy heuristic with small random perturbations. The greedy seeds help convergence at the beginning without removing too much diversity from the population.
- **Line 2:** Each chromosome is decoded into vehicle routes (Algorithm 2) and its cost is computed using the fuel-aware objective in Eq. (1).
- **Line 3:** The best chromosome found in the initial population is stored. This value will be updated whenever a better solution appears.
- **Line 4:** The main generational loop starts. It runs until either the maximum number of generations is reached or no improvement has been observed for a fixed number of generations in a row.
- **Line 5:** Elitism is applied: the top e individuals from the current population are copied unchanged into the new population. This protects the best solutions from being lost due to bad luck in crossover or mutation, which is a well-known issue in pure GA implementations.
- **Lines 6–10:** The new population is filled up by repeatedly producing offspring. Two parents are selected with tournament selection (line 7, see Algorithm 4). With probability p_c an Order Crossover (line 8,

Algorithm 5) is performed; otherwise, the offspring is a copy of the first parent. The offspring is then mutated with probability p_m (line 9, Algorithm 6) and added to the new population (line 10).

- **Line 11:** The current population is replaced by the new one.
- **Line 12:** All offspring are decoded and evaluated, and the global best *best* is updated if any of them is better.
- **Line 14:** The best solution found over all generations is returned.

A more detailed description of each operator, including separate pseudo codes, is given in the next section.

5 Solution Method

In this section we go through the implementation of the GA component by component. Each part is explained with its own pseudo code where useful, and the parameter choices are summarised at the end. A small toy example is also given in Section 5.11 to make the workflow easier to follow.

5.1 Chromosome Encoding

The chromosome is a permutation of the customer set $C = \{1, \dots, n\}$. The length of the chromosome is always n , and each gene is a customer ID. The order of customers in the chromosome represents the order in which they will be served. The depot is not explicitly stored in the chromosome — it is added during decoding.

This kind of encoding is sometimes called *indirect* encoding in the VRP literature, because the chromosome itself does not represent the vehicle routes directly. Instead, the routes are obtained from the chromosome through a deterministic decoding step. The main advantage of indirect encoding is that all standard permutation operators (Order Crossover, swap mutation, inversion mutation, and so on) can be applied without producing infeasible chromosomes.

5.2 Decoding: From Permutation to Vehicle Routes

The decoding step turns a chromosome into a set of vehicle routes. The procedure is a simple left-to-right scan: we open a route at the depot, add customers one by one until adding the next one would exceed Q , then close the route at the depot and open a new one. The pseudo code is given in Algorithm 2.

Algorithm 2 Permutation Decoder

```
1:  $routes \leftarrow \emptyset$ ,  $route \leftarrow [0]$ ,  $load \leftarrow 0$ 
2: for each customer  $c$  in chromosome (left to right) do
3:   if  $load + D_c \leq Q$  then
4:     Append  $c$  to  $route$ , set  $load \leftarrow load + D_c$ 
5:   else
6:     Append 0 to  $route$ , add  $route$  to  $routes$ 
7:      $route \leftarrow [0, c]$ ,  $load \leftarrow D_c$ 
8:   end if
9: end for
10: Append 0 to  $route$ , add  $route$  to  $routes$ 
11: return  $routes$ 
```

The behaviour of the decoder, line by line:

- **Line 1:** Three variables are initialised. $routes$ stores the list of completed routes, $route$ is the current route under construction (starting at the depot, node 0), and $load$ is the cumulative demand on the current vehicle.
- **Line 2:** The decoder iterates over the chromosome from left to right.
- **Lines 3–4:** If the current customer can be added to the active route without violating capacity, it is appended and the load is updated.
- **Lines 5–7:** Otherwise, the active route is closed at the depot and added to $routes$, and a new route is started with this customer as the first stop.
- **Line 10:** After the loop, the last open route is closed at the depot and added to $routes$.
- **Line 11:** The complete list of routes is returned.

Decoding is always feasible with respect to the capacity constraint because we close a route as soon as the next customer would not fit. The decoder is also intentionally simple: it just splits the permutation at the first point where capacity would be violated, without trying to be clever. This is sometimes called the *split-on-capacity* decoder, and it is a common choice in VRP-GA implementations because of its simplicity.

5.3 Fitness Evaluation

Once a chromosome is decoded into a set of routes, the fitness is computed using the objective function in Eq. (1). For each route, the load y_{ij} on arc (i, j) is equal to the remaining cargo on the vehicle at the time it traverses

the arc. The vehicle leaves the depot with the full load of the route, and the load decreases by D_c after each customer visit. This is why the same set of arcs travelled in a different order will, in general, give a different fuel cost — which is exactly the load-dependent effect that the GVRP captures.

If the number of routes k from the decoder exceeds the available fleet size m , a penalty of $\omega \cdot (k - m)$ is added to the objective, where ω is a large positive constant. In practice this rarely happens because m is set high enough in the experiments.

5.4 Initial Population

The initial population is built with two sources of diversity. About 80% of the individuals are random permutations of C , generated by shuffling the customer list. The remaining 20% are produced by a nearest-neighbor heuristic followed by a small number of random swaps. The nearest-neighbor heuristic starts at the depot and repeatedly picks the closest unvisited customer. Its pseudo code is given in Algorithm 3.

Algorithm 3 Nearest-Neighbor Greedy Heuristic

```

1:  $unvisited \leftarrow C$ ,  $perm \leftarrow []$ ,  $current \leftarrow 0$  (depot)
2: while  $unvisited \neq \emptyset$  do
3:    $next \leftarrow \arg \min_{c \in unvisited} d_{current, c}$ 
4:   Append  $next$  to  $perm$ 
5:   Remove  $next$  from  $unvisited$ 
6:    $current \leftarrow next$ 
7: end while
8: return  $perm$ 

```

Step 3 is the greedy choice that gives the heuristic its name — among the customers not yet visited, the one closest to the current location is picked. The result is a single permutation that the decoder of Section 5.2 can then split into vehicle routes. After generating a greedy chromosome, a small number of random swaps (about $n/5$ swaps) is applied so that the greedy seeds in the population are not all identical.

Mixing some greedy seeds with random ones tends to speed up convergence a bit, especially in the first generations, because the GA does not have to spend many iterations just to find a reasonable starting point. Using only greedy seeds would reduce diversity too much and the GA could get stuck in a local optimum early on, so the 80/20 split is a practical compromise that worked well in our tests. The same greedy heuristic is also used later as a baseline for comparison (Section 6.3).

5.5 Selection: Tournament

Parents are selected through k -ary tournament. In each tournament, k individuals are sampled uniformly at random from the population, and the one with the lowest fitness is chosen as a parent. The pseudo code is given in Algorithm 4.

Algorithm 4 Tournament Selection

- 1: Pick k distinct indices uniformly at random from $\{1, \dots, N_{pop}\}$
 - 2: $best_idx \leftarrow \arg \min_{i \in indices} F[i]$
 - 3: **return** $P[best_idx]$
-

The selection pressure can be tuned by changing k : a small k (e.g. 2) gives a relatively weak pressure and preserves diversity, while a larger k pushes the search toward the best individuals more aggressively. After some preliminary tests, $k = 3$ was used in all final experiments, which is a standard value in the GA literature and gives a reasonable balance between exploration and exploitation. The procedure is called twice in each generation to pick the two parents needed for crossover.

5.6 Crossover: Order Crossover (OX)

The crossover operator used is Order Crossover (OX), which is one of the most commonly used operators for permutation-based GAs. The procedure is given in Algorithm 5.

Algorithm 5 Order Crossover (OX)

- 1: Choose two cut points a, b uniformly at random with $a < b$
 - 2: $child \leftarrow [\text{NONE}, \dots, \text{NONE}]$ of length n
 - 3: $child[a..b] \leftarrow P_1[a..b]$ //copy segment from parent 1
 - 4: $used \leftarrow \{c : c \in child[a..b]\}$
 - 5: $write_pos \leftarrow 0$
 - 6: **for all** customers c in P_2 in order **do**
 - 7: **if** $c \notin used$ **then**
 - 8: **while** $write_pos$ falls inside the segment $[a, b]$ **do**
 - 9: $write_pos \leftarrow write_pos + 1$
 - 10: **end while**
 - 11: $child[write_pos] \leftarrow c$
 - 12: $write_pos \leftarrow write_pos + 1$
 - 13: **end if**
 - 14: **end for**
 - 15: **return** $child$
-

The first three lines copy a contiguous segment from parent 1 into the

same positions of the offspring. The rest of the chromosome is then filled by scanning parent 2 from left to right and inserting the customers that are not already in the segment, in the order they appear in parent 2. The while loop in lines 8–10 makes sure we skip the positions already taken by the segment.

The output is always a valid permutation, which is one of the main reasons OX is so commonly used in permutation-based GAs. The operator also has the practical advantage of preserving relative order from parent 2 outside the cut segment, so if parent 2 already has a good sub-sequence of customers, that information has a chance of being passed to the offspring even when the cut segment comes from parent 1. With probability $1 - p_c$, the crossover is skipped and a direct copy of parent 1 is used instead.

5.7 Mutation: Swap and Inversion

Two simple mutation operators are used:

- *Swap mutation*: pick two positions in the chromosome at random and exchange the two customers at those positions.
- *Inversion mutation*: pick two positions a and b at random and reverse the segment between them.

When mutation is applied to a chromosome (with probability p_m), one of the two operators is chosen with equal probability. The combined procedure is given in Algorithm 6.

Algorithm 6 Mutation Operator

```

1: Draw  $u_1 \sim \mathcal{U}(0, 1)$ 
2: if  $u_1 < p_m$  then
3:   Draw  $u_2 \sim \mathcal{U}(0, 1)$ 
4:   if  $u_2 < 0.5$  then
5:     Pick two distinct positions  $i, j$  uniformly at random //swap muta-
       tion
6:     Exchange  $perm[i]$  and  $perm[j]$ 
7:   else
8:     Pick two distinct positions  $a, b$  uniformly at random with  $a < b$ 
       //inversion mutation
9:     Reverse the segment  $perm[a..b]$ 
10:  end if
11: end if
12: return  $perm$ 

```

Swap mutation introduces small, local changes (only two positions affected), while inversion mutation can produce larger structural changes since

the entire segment between a and b is reversed. Using both keeps a reasonable balance between fine-grained adjustments (which are useful in the later generations when the population is already close to a good solution) and bigger jumps (which help escape local optima in the earlier generations). Both operators keep the chromosome a valid permutation, so we never end up with an infeasible offspring.

5.8 Elitism and Replacement

After the new population is built, the top e individuals from the previous generation are copied unchanged into the new one. This is the standard elitism technique and it makes sure that the best solution found so far cannot be lost due to bad luck in crossover or mutation. In our implementation $e = 2$, which is small enough not to harm the diversity but enough to preserve the current best.

5.9 Stopping Criterion

The algorithm stops in one of two cases:

- the maximum number of generations N_{gen} is reached, or
- the best fitness in the population has not improved for N_{stall} consecutive generations.

The second criterion is what is usually called *early stopping*. It is useful for small instances where the GA converges much earlier than N_{gen} , so we do not waste time running extra generations once the population has stabilised.

5.10 Parameter Choices

The values used in the final experiments are summarised in Table 1. They were chosen after a short set of preliminary runs on the smallest instances. We did not perform a full grid search because (i) the values in Table 1 are already within the typical ranges reported in the GA literature for permutation problems, and (ii) the algorithm was producing very low variance across runs from the start, which suggested the parameters were not at a problematic spot.

Parameter	Value	Description
N_{pop}	100	Population size
N_{gen}	200	Maximum number of generations
p_c	0.9	Crossover probability
p_m	0.15	Mutation probability
k	3	Tournament size
e	2	Elitism count
N_{stall}	50	Early stopping window
greedy ratio	0.20	Fraction of greedy seeds in initial population
ω	10^6	Penalty constant for excess vehicles

Table 1: GA parameter values used in the experiments.

5.11 A Toy Example

To make the workflow more concrete, Figure 1 shows how the algorithm works on a small 5-customer instance with capacity $Q = 50$. Panel (a) shows the instance: a depot in the middle of a 100×100 square and five customers with demands $\{10, 15, 8, 12, 20\}$. Panel (b) shows the chromosome representation, for example $[3, 1, 5, 2, 4]$. Panel (c) shows the decoded routes for this chromosome — the decoder splits the permutation into two vehicle routes because the cumulative demand on a single vehicle cannot exceed Q . Panel (d) shows the best solution found by the GA, which has chromosome $[5, 2, 3, 4, 1]$ and a fuel cost of 152.71, compared to 168.51 for the example permutation in panel (c).

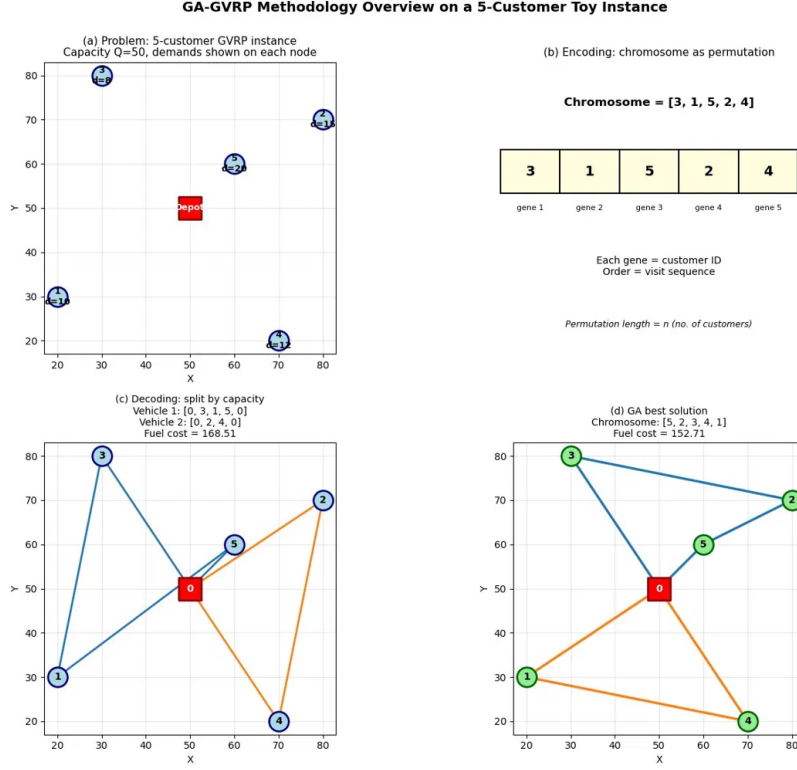


Figure 1: Workflow of the GA on a 5-customer GVRP instance. (a) Problem: depot and five customers with demands. (b) Encoding as a permutation chromosome. (c) Decoding the chromosome into vehicle routes, with fuel cost 168.51. (d) Best solution found by the GA, with fuel cost 152.71.

The same toy example also illustrates the load-dependent fuel effect that motivates the GVRP. Figure 2 shows two valid solutions for the same instance: a *naive* solution where customers are visited from the lightest demand to the heaviest, and the GA solution. Both solutions are feasible and visit all customers exactly once, but the GA solution has a fuel cost of 152.71 against 166.96 for the naive one — a difference of about 8.5% that comes purely from the visit order. The GA’s solution serves the heavier customers (e.g. customer 5 with demand 20) earlier, so the vehicle becomes lighter sooner and uses less fuel on the remaining arcs. This is exactly the kind of effect that Xiao et al. [1] discuss in their paper.

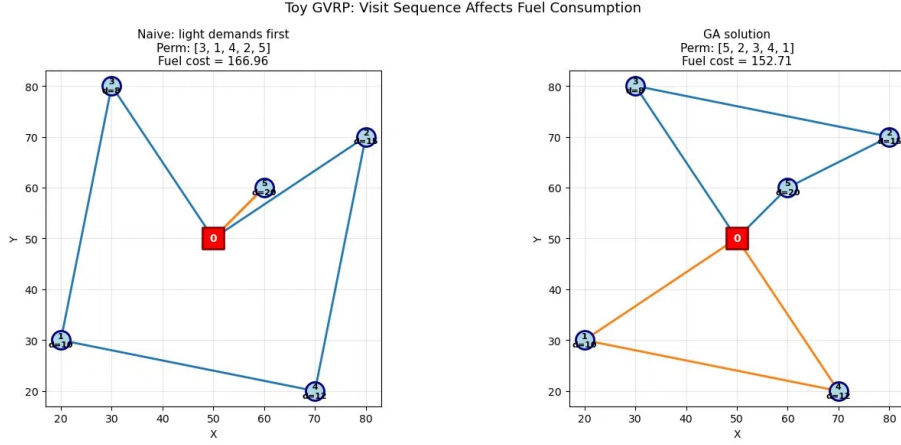


Figure 2: Sequence effect on the toy instance. *Left*: a naive permutation that visits customers in order of increasing demand, fuel cost 166.96. *Right*: the GA solution, fuel cost 152.71. Both solutions are feasible; the difference comes purely from the visit order.

6 Computational Study

6.1 Test Instances

Ten test instances were generated, ranging from $n = 10$ to $n = 150$ customers. Three small instances ($n \in \{10, 15, 20\}$), three medium ones ($n \in \{30, 40, 50\}$), and four large ones ($n \in \{75, 100, 125, 150\}$) were used in order to study the behaviour of the GA on different problem sizes, as suggested by the project guidelines.

The instances were generated following Christofides-style parameters [4]: customer coordinates are sampled uniformly at random in a $[0, 100] \times [0, 100]$ square, the depot is placed in the middle of this square, and customer demands are integers drawn uniformly from $[1, 30]$. Each instance was generated with a different random seed so that the customer distributions are not the same across instances. Distances are Euclidean. Vehicle capacities Q were chosen for each instance so that the average vehicle fills around 40–60%, which is in the typical range used in the VRP literature. Table 2 summarises the ten instances.

Name	n	Q	Total demand	Lower bound on vehicles
S1_n10	10	50	173	4
S2_n15	15	50	195	4
S3_n20	20	60	286	5
M1_n30	30	80	456	6
M2_n40	40	100	710	8
M3_n50	50	100	797	8
L1_n75	75	120	1120	10
L2_n100	100	150	1556	11
L3_n125	125	150	2090	14
L4_n150	150	200	2275	12

Table 2: The ten test instances used in the experiments. The lower bound on vehicles is $\lceil \text{total demand}/Q \rceil$.

For the fuel parameters, we used $\rho_0 = 0.165$ L/km and $\rho^* = 0.377$ L/km, which are representative values for a medium-duty diesel truck and are in the range used in the FCVRP literature [1]. The slope α is then computed from $\alpha = (\rho^* - \rho_0)/Q$. The fixed cost per vehicle was set to $F = 50$ and the unit fuel cost to $c_0 = 1.0$, which keeps the two cost terms in the same order of magnitude.

6.2 Experimental Setup

All experiments were run on a single machine with an Intel Core i7-11800H processor (8 cores, 16 logical threads at 2.30 GHz), 16 GB of RAM, and Windows 11 Pro 64-bit. The GA was implemented in Python 3.12.8 using only the standard library and NumPy for vectorised array operations. No external solver or GA framework was used — the algorithm was written from scratch.

Each instance was solved five times with different random seeds (specifically, seeds 1000 through 1004, so the experiments can be exactly reproduced). This is necessary because GA is a stochastic algorithm, so a single run does not say much about its real performance. From the five runs we report the best, mean, standard deviation, worst, coefficient of variation (CV, in percent), and the average wall-clock time per run.

For each instance we also computed two additional reference values:

- the cost of the deterministic nearest-neighbor greedy permutation (Algorithm 3, without the random swaps), which serves as a baseline;
- for the smallest instance (S1_n10), the exact optimum found by full enumeration of all $10! = 3,628,800$ permutations. This is only feasible for very small n , but it gives a true gap value to compare the GA against.

6.3 Computational Results

The main results are reported in Table 3. The GA finds solutions with low variability on every instance: the coefficient of variation is below 3.2% everywhere, and most instances are below 2%. This is a sign that the algorithm is stable in the sense that running it again with a different seed almost always gives a comparable solution.

Instance	n	Greedy	GA Best	GA Mean	Std	CV (%)	Imp. (%)	Time (s)
S1_n10	10	386.90	321.73	321.73	0.00	0.00	+16.8	0.13
S2_n15	15	383.51	329.48	333.06	3.46	1.04	+14.1	0.23
S3_n20	20	443.46	381.60	385.99	3.76	0.97	+13.9	0.34
M1_n30	30	538.04	476.00	488.60	13.83	2.83	+11.5	0.61
M2_n40	40	653.69	646.72	656.23	8.03	1.22	+1.1	0.81
M3_n50	50	748.90	716.20	725.35	6.01	0.83	+4.4	1.28
L1_n75	75	867.24	897.45	916.72	12.74	1.39	−3.5	1.74
L2_n100	100	975.73	1008.11	1050.37	24.61	2.34	−3.3	2.22
L3_n125	125	1349.74	1403.81	1441.41	21.57	1.50	−4.0	2.76
L4_n150	150	1101.46	1241.30	1307.60	41.11	3.14	−12.7	3.16

Table 3: Computational results over five GA runs per instance. Greedy is the cost of the deterministic nearest-neighbor solution. GA Best, Mean and Std are the corresponding statistics across the five runs. CV is the coefficient of variation in percent. Imp. is the relative improvement of GA Best over Greedy, in percent (positive = GA is better). Time is the average wall-clock time per GA run.

For the smallest instance (S1_n10), we also computed the true optimum by enumerating all $10! = 3,628,800$ permutations, which took about 51 seconds on the same machine. The optimum is 321.73, which is exactly the value the GA finds in every run, so the GA gap to optimum on this instance is 0.00%. We cannot do the same for the other instances because the number of permutations grows too fast (already $15! \approx 1.3 \times 10^{12}$), so for $n \geq 15$ we only have the greedy baseline as a reference.

A few other observations can be made from Table 3:

- *Small and medium instances:* the GA beats the greedy baseline by 11–17% on the smaller instances (S1–M1), and is still ahead but with a smaller margin on M2 and M3.
- *Large instances:* on L1 through L4, the greedy baseline is actually better than the GA. The gap is small for L1, L2 and L3 (between 3 and 4%) but grows to about 13% on L4.
- *Running times* stay short — even on the largest instance, one GA run takes only about three seconds. The total wall-clock time of the whole experiment (50 runs in total) was under three minutes.

The pattern in the Imp. column suggests that the pure GA, as implemented here, does not have enough exploitation power to beat a simple

route-construction heuristic on the larger instances. This is a known issue in the VRP-GA literature: the best-performing GAs for large VRPs almost always include a local search step (such as 2-opt or Or-opt) after crossover and mutation, as discussed by Vidal et al. [3]. We come back to this point in the discussion below.

One more thing that might look surprising at first is that instance L4_n150 has a lower best fitness (1241.30) than L3_n125 (1403.81), even though L4 has more customers. This is not a contradiction: the two instances are generated with different random seeds, so the coordinates and demand distributions are not directly comparable. Fuel cost depends on these factors as much as on the number of customers, so size alone is not a reliable predictor of cost across different random instances.

6.4 Convergence Behaviour

Figure 3 shows the convergence curves for all ten instances. Each subplot shows the five individual runs in grey and the mean curve in blue. A few observations can be made:

- For the smallest instances (S1, S2, S3), the GA converges within 20–50 generations and the early stopping criterion kicks in well before $N_{gen} = 200$. This is expected, since the search space is small and the GA can explore most of the high-quality solutions quickly.
- For the medium instances (M1, M2, M3), convergence is slower but still smooth, and most of the improvement happens in the first 100 generations. The curves then flatten out.
- For the large instances (L1–L4), the improvement is more gradual and continues throughout the 200 generations — there is no clear sign that the curves have flattened out by the end. This is consistent with the result of Section 6.3 that the GA underperforms greedy on these instances: it is still improving when we stop it, which suggests that either more generations or (more likely) a stronger local intensification step would be needed.

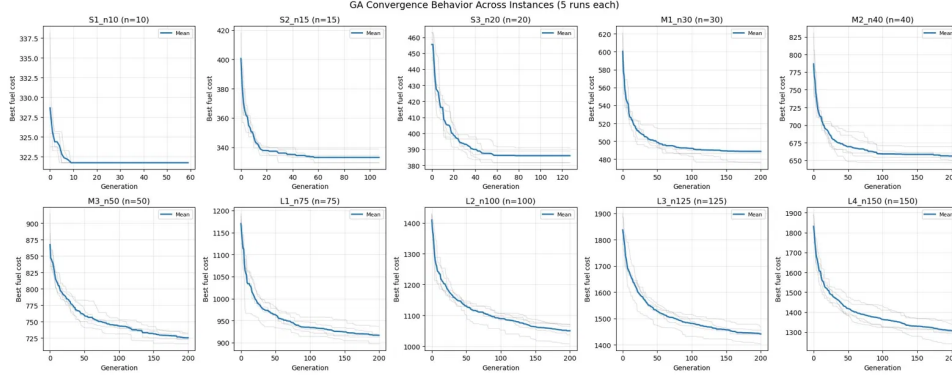


Figure 3: Convergence curves for all ten instances. Grey: individual runs; blue: mean of the five runs.

6.5 Discussion

There are two main things to take away from these experiments.

The first is positive: on the small and medium instances, the GA does a reasonable job. It finds the exact optimum on S1_n10 (verified by full enumeration), and beats the greedy baseline by double-digit percentages on most of the small and medium instances. The runs are also very consistent across seeds, with a coefficient of variation below 3.2% on every instance.

The second is more honest: on the large instances ($n \geq 75$), the pure GA implemented here is not enough to beat a simple greedy construction. This is not really a bug in the implementation — the GA does converge smoothly and the operators behave as expected. The issue is more about what a pure GA can and cannot do on this kind of problem. The crossover and mutation operators move customers around globally, but they do not refine the spatial structure of the routes; on large instances, the greedy nearest-neighbor heuristic already produces routes with a reasonable spatial layout, and our GA cannot improve on that just by reshuffling. This matches the general observation in the VRP literature [3] that competitive VRP metaheuristics almost always combine population-based search with a local search step.

It would also have been interesting to compare the GA against the exact MILP of Section 3 for the smaller instances. Solvers like Gurobi or CPLEX can solve the FCVRP MILP for n in the range of 10–20 in seconds, which would give true optimality gaps in the same way that brute force does for S1_n10. This was not done in the current project because of time constraints, but it would be a natural next step.

7 Conclusion

In this project a Genetic Algorithm was designed and implemented for the Green Vehicle Routing Problem with load-dependent fuel consumption, using the MILP formulation of Xiao et al. [1] as the reference model. The algorithm uses permutation-based encoding with a split-on-capacity decoder, a mix of random and greedy initial individuals, tournament selection, Order Crossover, swap and inversion mutation, and elitism. A small toy example with 5 customers was used to illustrate the workflow and the load-dependent fuel effect, and the algorithm was then tested on ten instances ranging from 10 to 150 customers, with five runs per instance and a deterministic greedy baseline for comparison.

The most important results are the following. The GA produces low-variance solutions across runs (coefficient of variation under 3.2% on every instance), runs in just a few seconds even for the largest instance, and converges smoothly on all problem sizes. On the smallest instance, the GA finds the exact optimum (verified by enumerating all $10!$ permutations). On the small and medium instances ($n \leq 50$), the GA beats the greedy nearest-neighbor baseline by between 1% and 17%. On the larger instances ($n \geq 75$), however, the greedy baseline is actually better than the GA — the gap reaches about 13% on the largest instance ($n = 150$). This is a known limitation of pure GA without local search, and it is consistent with what the VRP literature reports.

The most obvious next step would therefore be to add a local search procedure (2-opt or Or-opt being the standard choices) inside the GA loop, which would almost certainly improve the results on the large instances. Other natural extensions are: solving the MILP on the smaller instances with an exact solver like Gurobi or CPLEX to obtain true optimality gaps for n up to roughly 20; testing the algorithm on standard benchmark sets such as the Christofides CMT instances or the Augerat A-set, adapted to the GVRP setting following Xiao et al. [1]; and experimenting with different fuel parameter values to see how sensitive the routes are to the FCR slope α .

Code Availability. The complete Python implementation, generated test instances, experimental outputs, and instructions required to reproduce the computational study are publicly available at

<https://github.com/mrehaozdemir/EMU676-GVRP-GA>.

The repository includes the source code, generated figures, sample data, and a README file describing how to reproduce the reported experiments.

References

- [1] Xiao, Y., Zhao, Q., Kaku, I., and Xu, Y.
Development of a fuel consumption optimization model for the capacitated vehicle routing problem.
Computers & Operations Research, 39(7):1419–1431, 2012.
- [2] Asghari, M. and Mirzapour Al-e-hashem, S.M.J.
Green vehicle routing problem: A state-of-the-art review.
International Journal of Production Economics, 231:107899, 2021.
- [3] Vidal, T., Laporte, G., and Matl, P.
A concise guide to existing and emerging vehicle routing problem variants.
European Journal of Operational Research, 286(2):401–416, 2020.
- [4] Christofides, N., Mingozzi, A., and Toth, P.
The vehicle routing problem.
In: Combinatorial Optimization, pages 315–338. Wiley, Chichester, 1979.
- [5] Bektaş, T. and Laporte, G.
The Pollution-Routing Problem.
Transportation Research Part B: Methodological, 45(8):1232–1250, 2011.
- [6] Demir, E., Bektaş, T., and Laporte, G.
A review of recent research on green road freight transportation.
European Journal of Operational Research, 237(3):775–793, 2014.
- [7] Lin, C., Choy, K.L., Ho, G.T.S., Chung, S.H., and Lam, H.Y.
Survey of Green Vehicle Routing Problem: Past and future trends.
Expert Systems with Applications, 41(4):1118–1138, 2014.
- [8] Erdoğan, S. and Miller-Hooks, E.
A Green Vehicle Routing Problem.
Transportation Research Part E: Logistics and Transportation Review, 48(1):100–114, 2012.
- [9] Holland, J.H.
Adaptation in Natural and Artificial Systems.
University of Michigan Press, Ann Arbor, 1975.